

CSE2315 — Assignment 7

Vlad Paun
6152937

March 26, 2026

Exercise 1

Prove that the following problem is in NP:

$\{(V, E) \mid \text{the graph } G = (V, E) \text{ contains a simple path } \pi \text{ such that } \forall v \in V : v \in \pi\}$

In a different notation we would write:

Name: Hamiltonian Cycle (HAMC)

Input: A graph $G = (V, E)$.

Question: Is there a simple cycle π in G such that all vertices $v \in V$ are in that cycle?

Proof. The following is a verifier V for *HAMC*:

$V =$ "On input $\langle G, c \rangle$:

1. Test whether c is a tuple containing all nodes in G exactly once.
2. For each i between 1 and $m - 1$ (where m is the number of nodes in G): check that there is an edge in G from node c_i to c_{i+1}
3. Check that there is an edge in G from c_m to c_1

Stages 1. and 3. run exactly once, and stage 2. runs m times, so there is a polynomial amount of stages. We can easily see that each stage can be completed in polynomial time, so the verifier is overall polynomial. Therefore, *HAMC* $\in$ *NP* $\square$

Exercise 2

Old exam question, endterm 23–24:

For the following claim, if it is true: prove it. If it is false, give an explanation showing the claim is false and if possible a counterexample. Remember that the cover of our exam reads: "Unless otherwise stated we assume that $P \neq NP$ and that $NP \neq EXP$."

For three languages A, B, C if $A \leq_P B$ and $B \leq_m C$ with $C \in NPC$. Furthermore we know that there is a regular expression R such that $L(R) = A$. Then $B \in P$.

False. Take for example $A = \emptyset$, which is regular, and $B = C = SAT$. A is karp reducible to B by a function that takes any input ignores it and outputs $p \wedge \bar{p}$. B is equal to C so it is mapping reducible to it by a function that does nothing. And finally $C \in NPC$. So all 3 conditions hold. However, $B = SAT \in NP$ and we assume $P \neq NP$.

Exercise 3

Old exam question:

On the (famous?) British Quiz Show “The University Challenge”, the following question was asked in the grand final of 2018/2019.

... which problem in computing asks whether all problems with a solution verifiable in finite time, can also be solved in finite time.

The contestants answer with “P equals NP” and the host answers back with “Yes that’s right, P versus NP, yes”.

Assuming this is the answer the host was looking for, what is wrong with the question he asks?

Answer: The question asks about finite time, which is trivial. The correct question should ask about solutions verifiable and solvable in polynomial time.

Exercise 4

Old exam question, resit 20–21:

(a) **Prove that the following problem is in NP.**

Name: AmazingCustomChallenge

Instance: Given a set L of lectures, a set of students S , a collection of subsets T with $|T| = |S|$ so that T_i is the set of lectures that student i needs to attend, and an integer K .

Question: Is there a function $f : L \rightarrow \{1, 2, \dots, K\}$ that maps lectures to time slots so that all students can attend all their lectures. In other words, is there a function so that there is no student who has to attend two lectures that are mapped to the same time slot.

Proof. The following is a NTM N that decides AmazingCustomChallenge in nondeterministic polynomial time:

$N =$ "On input $\langle L, S, T, K \rangle$:

1. For each lecture $l \in L$, nondeterministically choose a value $f(l) \in \{1, \dots, K\}$
2. For every student i check that no lectures in $T_i = \{l_{i_1}, \dots, l_{i_x}\}$ are assigned the same time-slot by looking them up for every pair (l_{i_m}, l_{i_n})
3. Accept if no conflict is found, reject otherwise

Stage one has to guess $|L|$ numbers, and writing down each number would take $O(\log K)$, so overall stage 1 takes $O(|L| \log K)$. Stage 2 needs to check all pairs of lectures in every T_i , so this would be bounded by $O(\sum_{i=1}^{|S|} |T_i|^2)$, which would be a worst case of $O(|S| \cdot |L|^2)$. Stage 3 is constant. Therefore, overall machine N runs in $O(|L| \log K + |S| \cdot |L|^2)$, which is polynomial in the size of the input. $\square$

- (b) Not originally part of the exam question. Your answer to the previous question should have included time complexities. If we are to describe them in terms of $|I|$ where I is an instance of the problem, what would be the complexity? (Hint: first think about $|I|$, is it $|S|$, or $|L|$, or some combination? How does K factor into this?)

$$|I| \in O(|L| + |S| + \sum_i |T_i| + \log K)$$

$$|I| \in O(|L| + |S| + |S| \cdot |L| + \log K)$$

So, overall the runtime could be expressed roughly as $T(I) \in O(|I|^3)$.

Exercise 5

Variation on old exam question, resit 20–21:

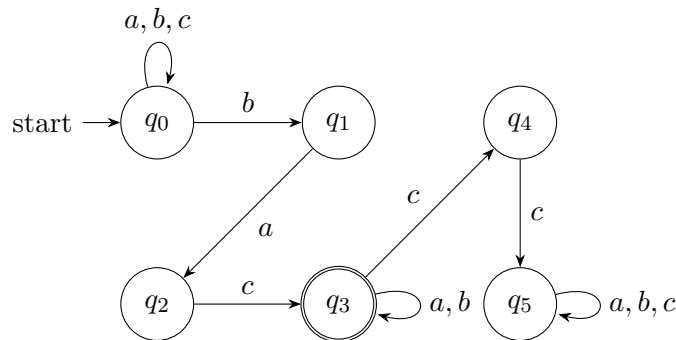
Show the claim is true or false, starting your answer with either “True” or “False”.

Given two problems A and B that can be solved using a PDA, it is possible to create a Karp-reduction from A to B and vice versa.

False. Take for example $A = \Sigma^*$ and $B = \emptyset$. Suppose there would exist a Karp reduction from A to B , then there exists a computable function $f : \Sigma^* \rightarrow \Sigma^*$ such that $x \in A \iff f(x) \in B$. However, in our example $\forall x \in \Sigma^* : x \in A$. So $\forall x \in \Sigma^* : f(x) \in B$, but B must be empty. Therefore, no such function can exist.

Exercise 6

Revision, old exam question, resit 20–21: Given this NFA N , give a regular expression R so that $L(N) = L(R)$.



$$(a \cup b \cup c)^* bac(a \cup b)^*$$

Exercise 7

Revision, old exam question, endterm 16–17: Suppose we have the following language:

$$L = \{\langle M, v, w \rangle \mid M \text{ outputs } w \text{ on the tape when run on input } v\}$$

Use a mapping reduction to prove that L is undecidable. Give:

(a) A suitable problem to reduce from.

We will reduce from $HALT_{TM}$

(b) A suitable reduction function $f : \Sigma^* \rightarrow \Sigma^*$.

Take $f : \langle M, v \rangle \rightarrow \langle M', v, vladulon \rangle$, where:

$M' =$ "on input x :

1. Ignore input x .
2. Simulate M on v and ignore the output.
3. Output *vladulon* on the tape"

(c) A proof showing f satisfies the requirements of a mapping reduction.

Proof. We must show that:

1. f is computable
2. $\forall \langle M, v \rangle : \langle M, v \rangle \in HALT_{TM} \iff f(\langle M, v \rangle) \in L$

For computability, we can trivially see that we can construct the description of M' to just simulate M on the hardcoded input v and then write our specific *vladulon* string to the tape, and then after that description, append our encoding for v and *vladulon*.

For Correctness, take any input $\langle M, v \rangle$ and consider the 2 cases:

1. $\langle M, v \rangle \in HALT_{TM}$: then M halts on v ; thus, by construction M' will simulate M on v and eventually halt and after it does so it will write *vladulon* on the tape. the output of our function f says that M' will correctly output *vladulon* on the tape when ran on input v , which it does, so $\langle M', v, vladulon \rangle \in L$.
2. $\langle M, v \rangle \notin HALT_{TM}$; then, M does not halt when ran on v . So neither will our machine M' when it simulates M on v . So M' will get stuck in a loop before it can print *vladulon* on the tape. Therefore M' does not output *vladulon* when ran on v , so $\langle M', v, vladulon \rangle \notin L$.

□